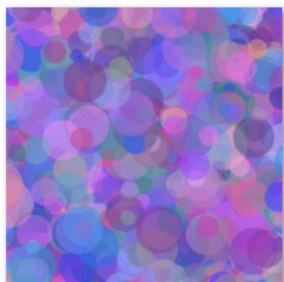


以下是斯坦福 cs149 《并行计算》课程[第三次作业](#)的文档翻译。在这份翻译中，代码和链接等等得到了保留，而代码中的注释、表格中的表头以及程序使用方式等等内容均完成了翻译。这份翻译基于以下提交 `fd0f70543f5dbe689b9bf8231ff385dfd35be097` 中的 `README.md` 文件。

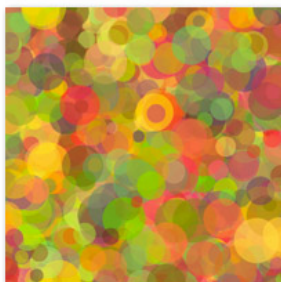
作业 3：一个简单的 CUDA 渲染器

截止日期：11月8日星期三，美国太平洋标准时间晚上11:59

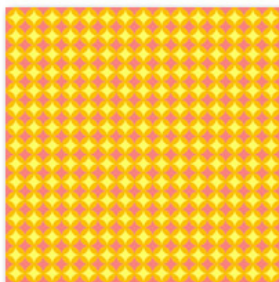
总分：100分



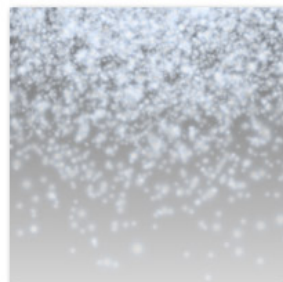
Random 10K



Random 100K



Pattern



Snow

概述

在这个作业中，你将编写一个在 CUDA 上运行的并行渲染器，用于绘制彩色圆圈。虽然这个渲染器非常简单，但并行化渲染器需要你设计和实现能够在并行中高效构建和操作的数据结构。这是一个具有挑战性的作业，所以建议你尽早开始。**真的，建议你尽早开始。**祝你好运！

环境设置

- 你将在 Amazon Web Services (AWS) 上的支持 GPU 的虚拟机上收集结果（即运行性能测试）。请按照 [AWS 设置说明](#) 中的说明设置运行作业的机器。
- 使用以下命令从课程的 Github 下载作业的起始代码：

```
git clone https://github.com/stanford-cs149/asst3
```

CUDA C 程序员指南的 [PDF 版本](#) 或 [网页版本](#) 是学习如何编写 CUDA 程序的极好参考。网上有丰富的 CUDA 教程和 SDK 示例（只需谷歌搜索！）以及 [NVIDIA 开发者网站](#) 上的资源。特别是，你可能会喜欢免费的 Udacity 课程 [CUDA 并行编程入门](#)。

[CUDA C 编程指南](#) 中的表格 G.1 是一个方便的参考，列出了 NVIDIA T4 GPU 在本次作业中支持的每个线程块的最大 CUDA 线程数、线程块大小、共享内存等信息。NVIDIA T4 GPU 支持 CUDA 计算能力 7.5。

对于 C++ 问题（例如 `virtual` 关键字的含义），[C++ 超级常见问题解答](#) 是一个很好的资源，以详细且易于理解的方式解释了这些问题（不像许多 C++ 资源），而且由 C++ 的创建者 Bjarne Stroustrup 共同编写！

警告

为了节省资源，当 CPU 活动低于 2% 并持续 15 分钟后，虚拟机将自动停止。

这意味着，如果你不进行 CPU 密集型工作（如编写代码），虚拟机将会关闭。

因此，我们建议你在本地开发代码，然后手动将代码复制到机器上，或者使用 git 将你的提交拉到虚拟机上。使用 git 很好，因为你可以返回到代码的以前版本。

如果你以前没有设置过私有 git 仓库，以下是一些帮助你入门的资源。确保 github 仓库是私有的，以确保你没有违反荣誉守则。

设置 git 的有用链接：

- [添加远程仓库](#) 以连接到你的私有仓库。
- [添加 ssh 密钥](#) 以设置 ssh 密钥。我们建议使用无密码和默认名称 id_rsa 的方式进行设置。

一旦你有了 ssh 密钥并知道如何连接到远程仓库，你需要做以下两件事来设置你的环境。

1. 将你的私钥复制到服务器的 .ssh 文件夹中 (id_rsa 在你的 .ssh 文件中)
2. 在服务器和本地创建一个名为 config 的文件，内容如下。

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_rsa
```

你现在应该能够从服务器和本地拉取和推送提交了！

第 1 部分：CUDA 热身 1：SAXPY (5 分)

为了让你熟悉编写 CUDA 程序，你的热身任务是用 CUDA 重新实现作业 1 中的 SAXPY 函数。本部分作业的起始代码位于作业代码库的 `/saxpy` 目录中。你可以通过在 `/saxpy` 目录中运行 `make` 和 `./cudaSaxpy` 来构建和运行 saxpy CUDA 程序。

请完成 `saxpy.cu` 中 `saxpyCuda` 函数的实现。你需要分配设备全局内存数组，并在执行计算之前将主机输入数组 `x`、`y` 和 `result` 的内容复制到 CUDA 设备内存中。在 CUDA 计算完成后，结果必须复制回主机内存。请参阅程序员指南第 3.2.2 节（网页版本）中 `cudaMemcpy` 函数的定义，或查看作业起始代码中指向的有用教程。

作为实现的一部分，请在 `saxpyCuda` 中的 CUDA 内核调用周围添加计时器。在你添加后，你的程序应该计时两次执行：

- 提供的起始代码包含计时器，测量从 **将数据复制到 GPU、运行内核到将数据复制回 CPU** 的整个过程的时间。
- 你还应该插入计时器，测量 **仅内核执行的时间**。（不应包括 CPU 到 GPU 的数据传输时间或从 GPU 返回 CPU 的结果传输时间。）

在添加后者的计时代码时需要注意：默认情况下，CUDA 内核在 GPU 上的执行与在 CPU 上运行的主应用程序线程是 **异步** 的。例如，如果你编写如下代码：

```
double startTime = CycleTimer::currentSeconds();
saxpy_kernel<<<blocks, threadsPerBlock>>>>(N, alpha, device_x, device_y,
device_result);
double endTime = CycleTimer::currentSeconds();
```

你会测得一个惊人快的内核执行时间！（因为你只是计时 API 调用本身的成本，而不是实际在 GPU 上执行计算的成本。）

因此，你需要在内核调用后插入 `cudaDeviceSynchronize()` 调用，以等待 GPU 上所有 CUDA 工作的完成。这个 `cudaDeviceSynchronize()` 调用会在 GPU 上所有之前的 CUDA 工作完成后返回。请注意，在我们使用的条件下，`cudaMemcpy()` 是同步的，因此在确保内存传输到 GPU 完成后，不需要在 `cudaMemcpy()` 之后调用 `cudaDeviceSynchronize()`。（想了解更多信息，请参阅[此文档](#)。）

```
double startTime = CycleTimer::currentSeconds();
saxpy_kernel<<<blocks, threadsPerBlock>>>(N, alpha, device_x, device_y,
device_result);
cudaDeviceSynchronize();
double endTime = CycleTimer::currentSeconds();
```

请注意，在包含从 CPU 到 GPU 数据传输时间的测量中，在最终计时器（在调用将数据返回到 CPU 的 `cudaMemcpy()` 后）之前不需要调用 `cudaDeviceSynchronize()`，因为 `cudaMemcpy()` 在传输完成之前不会返回到调用线程。

问题 1. 与基于顺序 CPU 的 SAXPY 实现（回想作业 1 中程序 5 的结果）相比，你观察到的性能如何？

问题 2. 比较并解释由两组计时器提供的结果之间的差异（仅内核执行计时与包括数据移动到 GPU 和返回的整个过程计时）。观察到的带宽值是否与机器不同组件的报告带宽值大致一致？（你应该使用网络来查找 NVIDIA T4 GPU 的内存带宽。提示：<https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf>。AWS 的内存总线的预期带宽是 4 GB/s，这与 16 通道 [PCIe 3.0](#) 不匹配。多种因素阻止了峰值带宽，包括 CPU 主板芯片组性能以及是否使用了“固定”主机 CPU 内存作为传输的源——后者允许 GPU 直接访问内存而无需经过虚拟内存地址转换。如果你有兴趣，可以在这里找到更多信息：<https://kth.instructure.com/courses/12406/pages/optimizing-host-device-data-communication-i-pinned-host-memory>。）

第 2 部分：CUDA 热身 2：并行前缀和(10 分)

现在你已经熟悉了 CUDA 程序的基本结构和布局，作为第二个练习，你需要提出函数 `find_repeats` 的并行实现。该函数在给定一个整数列表 `A` 时，返回一个所有满足 `A[i] == A[i+1]` 的索引 `i` 的列表。

例如，给定数组 `{1,2,2,1,1,1,3,5,3,3}`，你的程序应输出数组 `{1,3,4,8}`。

独占前缀和

我们希望你通过首先实现并行独占前缀和操作来实现 `find_repeats`。

独占前缀和接受一个数组 `A` 并生成一个新数组 `output`，在每个索引 `i` 处，该数组包含直到但不包括 `A[i]` 的所有元素的和。例如，给定数组 `A={1,4,6,8,2}`，独占前缀和的输出为 `output={0,1,5,11,19}`。

以下“类 C”代码是扫描的迭代版本。在伪代码中，我们使用 `parallel_for` 表示可能的并行循环。这与我们在课堂上讨论的算法相同：http://cs149.stanford.edu/fall23/lecture/dataparallel/slide_17

```
void exclusive_scan_iterative(int* start, int* end, int* output) {

    int N = end - start;
    memmove(output, start, N*sizeof(int));

    // 上扫阶段
    for (int two_d = 1; two_d <= N/2; two_d*=2) {
        int two_dplus1 = 2*two_d;
        parallel_for (int i = 0; i < N; i += two_dplus1) {
            output[i+two_dplus1-1] += output[i+two_d-1];
        }
    }

    output[N-1] = 0;

    // 下扫阶段
```

```

    for (int two_d = N/2; two_d >= 1; two_d /= 2) {
        int two_dplus1 = 2*two_d;
        parallel_for (int i = 0; i < N; i += two_dplus1) {
            int t = output[i+two_d-1];
            output[i+two_d-1] = output[i+two_dplus1-1];
            output[i+two_dplus1-1] += t;
        }
    }
}

```

我们希望你使用此算法在 CUDA 中实现并行前缀和的版本。你必须在 `scan/scan.cu` 中实现 `exclusive_scan` 函数。你的实现将包括主机代码和设备代码。该实现将需要多个 CUDA 内核启动（每个并行循环一个内核启动）。

注意：在起始代码中，上述参考解决方案扫描实现假定输入数组的长度（`N`）是 2 的幂。在 `cudaScan` 函数中，我们通过在 GPU 上分配相应缓冲区时将输入数组长度舍入到下一个 2 的幂来解决此问题。然而，代码只从 GPU 缓冲区复制 `N` 个元素回到 CPU 缓冲区。这一事实应该简化你的 CUDA 实现。

编译生成二进制文件 `cudaScan`。命令行使用说明如下：

Usage: `./cudaScan [options]`

程序选项：

<code>-m --test <TYPE></code>	在输入上运行指定的函数。有效的测试有： <code>scan</code> ， <code>find_repeats</code> （默认： <code>scan</code> ）
<code>-i --input <NAME></code>	在给定的输入类型上运行测试。有效的输入有： <code>ones</code> ， <code>random</code> （默认： <code>random</code> ）
<code>-n --arraysize <INT></code>	数组中的元素数量
<code>-t --thrust</code>	使用 <code>Thrust</code> 库实现
<code>-? --help</code>	显示此信息

使用前缀和实现“查找重复项”

一旦你编写了 `exclusive_scan`，请在 `scan/scan.cu` 中实现 `find_repeats` 函数。这将涉及编写更多的设备代码，以及对 `exclusive_scan()` 的一次或多次调用。你的代码应将重复元素的列表写入提供的输出指针（在设备内存中），然后返回输出列表的大小。

调用你的 `exclusive_scan` 实现时，请记住 `start` 数组的内容将复制到 `output` 数组中。另外，传递给 `exclusive_scan` 的数组应位于 `device` 内存中。

评分：我们将测试你的代码在随机输入数组上的正确性和性能。

作为参考，下面提供了一个扫描得分表，显示了在 K80 GPU 上简单 CUDA 实现的性能。要检查你的 `scan` 和 `find_repeats` 实现的正确性和性能得分，请分别运行 `./checker.pl scan` 和 `./checker.pl find_repeats`。这样做将产生如下所示的参考表；你的得分完全基于你代码的性能。为了获得满分，你的代码性能必须在提供的参考解决方案的 20% 以内。

扫描得分表:

元素数量	参考时间	学生时间	得分
1000000	0.766	0.143 (F)	0
10000000	8.876	0.165 (F)	0
20000000	17.537	0.157 (F)	0
40000000	34.754	0.139 (F)	0
		总得分:	0/5

作业的这一部分主要是为了让你更多地练习编写 CUDA 代码并以数据并行的方式思考，而不是为了性能调优。要在这一部分获得满分的性能分，不需要（或几乎不需要）性能调优，只需将算法伪代码直接移植到 CUDA。然而，有一个技巧：扫描的天真实现可能会在伪代码中的每次并行循环迭代中启动 N 个 CUDA 线程，并在内核中使用条件执行来确定哪些线程实际上需要工作。这样的解决方案不会高效！（考虑到上扫阶段最后一层循环迭代，其中只有两个线程会工作！）一个满分的解决方案只会为最内层并行循环的每次迭代启动一个 CUDA 线程。

测试工具：默认情况下，测试工具运行一个伪随机生成的数组，该数组每次运行程序时都是相同的，以帮助调试。你可以传递参数 `-i random` 在随机数组上运行测试 - 我们在评分时会这样做。我们鼓励你为你的程序提出替代输入，以帮助你评估它。你还可以使用 `-n <size>` 选项更改输入数组的长度。

参数 `--thrust` 将使用 [Thrust 库](#) 的 [独占扫描](#) 实现。**任何能够创建与 Thrust 竞争的实现的的同学最多可以获得两分的额外加分。**

第 3 部分：一个简单的圆形渲染器 (85 分)

现在进入真正的展示！

作业起始代码的 `/render` 目录包含了一个绘制彩色圆圈的渲染器实现。构建代码，并使用以下命令行运行渲染器：`./render -r cpuref rgb`。程序将输出一个包含三个圆圈的图像 `output_0000.ppm`。现在使用命令行 `./render -r cpuref snow` 运行渲染器。此时输出图像将是下雪的场景。PPM 图像可以直接在 OSX 上通过预览查看。对于 Windows，你可能需要下载一个查看器。

注意：你也可以使用 `-i` 选项将渲染器输出发送到显示器而不是文件。（在下雪的情况下，你会看到下雪的动画。）但是，要使用交互模式，你需要能够设置 X-windows 转发到你的本地机器。（[这个参考](#) 或 [这个参考](#) 可能会有所帮助。）

作业起始代码包含两个版本的渲染器：一个顺序的、单线程的 C++ 参考实现，在 `refRenderer.cpp` 中实现；以及一个不正确的并行 CUDA 实现，在 `cudaRenderer.cu` 中实现。

渲染器概述

我们鼓励你通过检查 `refRenderer.cpp` 中的参考实现来熟悉渲染器代码库的结构。在渲染第一帧之前，将调用 `setup` 方法。在你的 CUDA 加速渲染器中，这个方法可能包含你所有的渲染器初始化代码（分配缓冲区等）。每帧都调用 `render`，负责将所有圆圈绘制到输出图像中。渲染器的另一个主要函数 `advanceAnimation` 也在每帧调用一次。它更新圆圈的位置和速度。在这个作业中，你不需要修改 `advanceAnimation`。

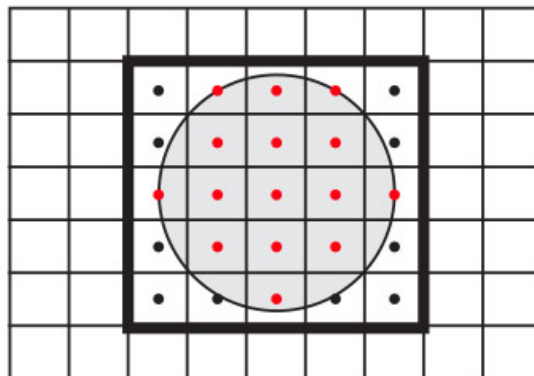
渲染器接受一个圆圈数组（3D 位置、速度、半径、颜色）作为输入。渲染每一帧的基本顺序算法是：

```

清空图像
对每个圆圈
    更新位置和速度
对每个圆圈
    计算屏幕边界框
    对边界框中的所有像素
        计算像素中心点
        如果中心点在圆圈内
            计算圆圈在该点的颜色
            将圆圈的贡献混合到该像素的图像中

```

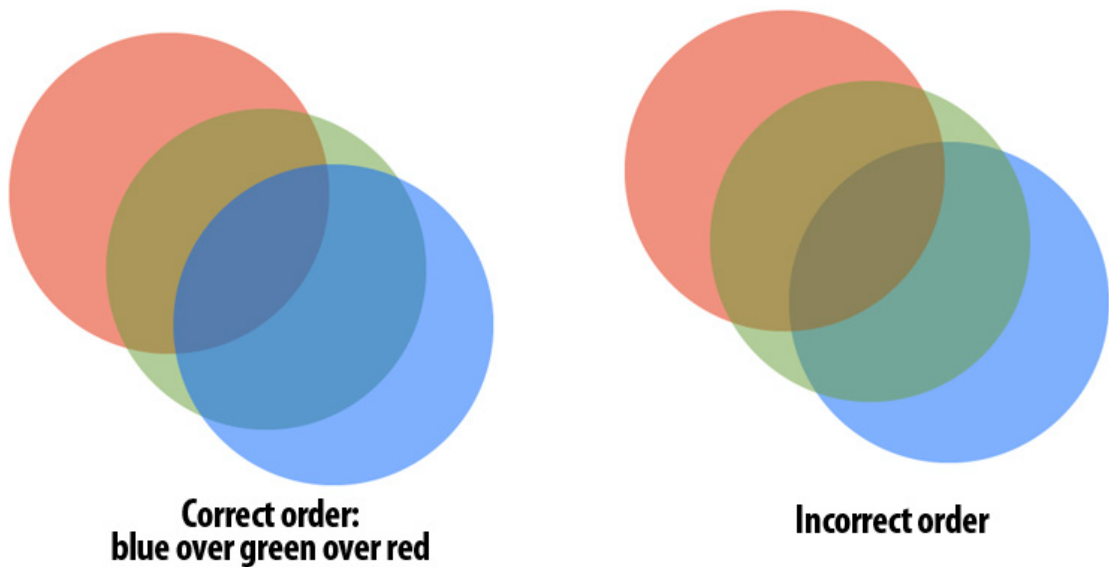
下图说明了使用点在圆圈内测试计算圆圈像素覆盖的基本算法。请注意，只有当像素的中心位于圆圈内时，圆圈才会对输出像素贡献颜色。



渲染器的一个重要细节是它渲染 **半透明** 的圆圈。因此，任何一个像素的颜色不是单个圆圈的颜色，而是所有重叠该像素的半透明圆圈的贡献混合结果（注意上面伪代码中的“混合贡献”部分）。渲染器通过一个包含红色（R）、绿色（G）、蓝色（B）和不透明度（alpha）值的四元组（RGBA）表示圆圈的颜色。Alpha = 1 对应于完全不透明的圆圈。Alpha = 0 对应于完全透明的圆圈。为了在一个颜色为 (P_r, P_g, P_b) 的像素上绘制一个颜色为 $(C_r, C_g, C_b, C_{alpha})$ 的半透明圆圈，渲染器使用以下公式：

$$\begin{aligned}
 result_r &= C_alpha * C_r + (1.0 - C_alpha) * P_r \\
 result_g &= C_alpha * C_g + (1.0 - C_alpha) * P_g \\
 result_b &= C_alpha * C_b + (1.0 - C_alpha) * P_b
 \end{aligned}$$

注意，组合不是交换的（对象 X 在 Y 上与对象 Y 在 X 上看起来不一样），因此渲染器必须按照应用程序提供的顺序绘制圆圈。（你可以假设应用程序按深度顺序提供圆圈。）例如，考虑下图，左边的图像中蓝色圆圈绘制在绿色圆圈之上，绿色圆圈绘制在红色圆圈之上。在右边的图像中，圆圈以不同的顺序绘制，输出图像看起来不正确。



CUDA 渲染器

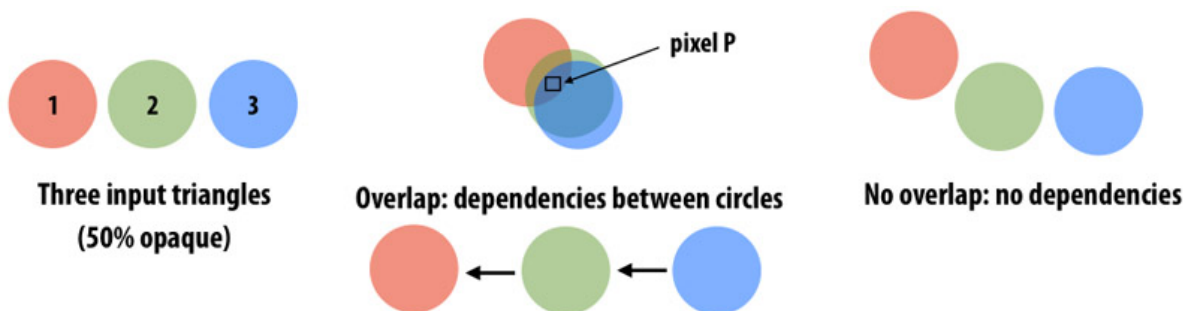
在熟悉了参考代码中实现的圆圈渲染算法后，现在研究 `cudaRenderer.cu` 中提供的渲染器的 CUDA 实现。你可以使用 `--renderer cuda` (或 `-r cuda`) `cuda` 程序选项运行 CUDA 实现的渲染器。

提供的 CUDA 实现通过所有输入圆圈并行化计算，为每个 CUDA 线程分配一个圆圈。虽然这个 CUDA 实现是一个完整的圆圈渲染器的数学实现，但它包含几个你需要在这个作业中修复的重大错误。具体来说：当前实现不确保图像更新是原子操作，并且不保留图像更新的所需顺序（下面将描述顺序要求）。

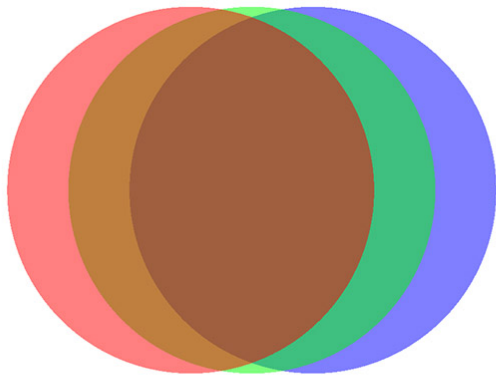
渲染器要求

你的并行 CUDA 渲染器实现必须保持顺序实现中简单保留的两个不变量。

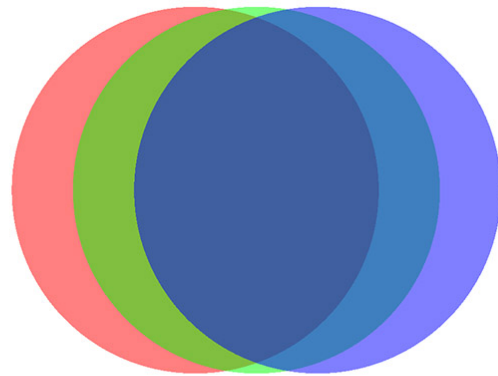
1. **原子性：** 所有图像更新操作必须是原子的。临界区包括读取四个 32 位浮点值（像素的 rgba 颜色）、将当前圆圈的贡献与当前图像值混合，然后将像素的颜色写回内存。
2. **顺序：** 你的渲染器必须按 **圆圈输入顺序** 执行对图像像素的更新。也就是说，如果圆圈 1 和圆圈 2 都对像素 P 有贡献，则任何由于圆圈 1 对 P 的图像更新必须在圆圈 2 对 P 的更新之前应用到图像上。如上所述，保留顺序要求允许正确渲染透明圆圈。（它对图形系统还有许多其他好处。如果好奇，请与 Kayvon 交谈。）**一个关键的观察是，顺序的定义只指定对同一像素的更新顺序。** 因此，如下图所示，不对不贡献相同像素的圆圈有顺序要求。这些圆圈可以独立处理。



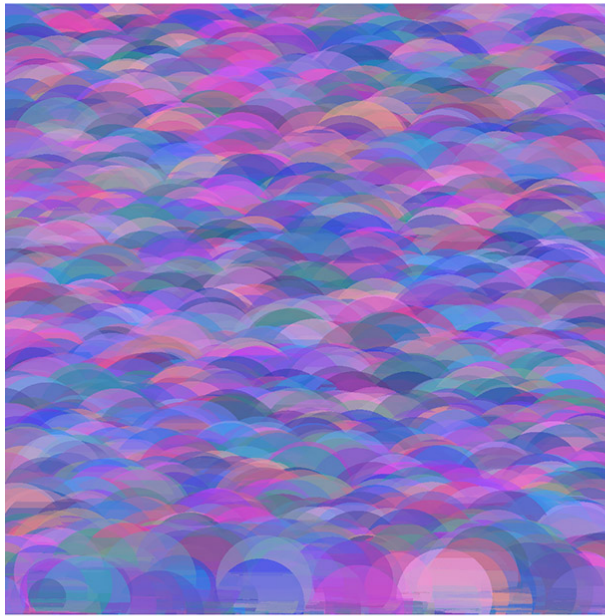
由于提供的 CUDA 实现不满足这些要求之一或全部，通过运行 CUDA 渲染器实现的 rgb 和圆圈场景，可以看到不正确地遵守顺序或原子性的结果。你会看到生成的图像中有水平条纹，如下图所示。这些条纹会随着每一帧而变化。



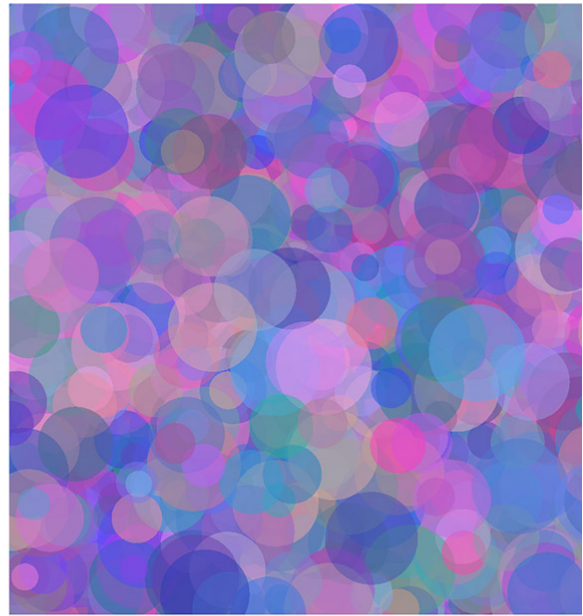
Incorrect (RGB)



Correct (RGB)



Incorrect (rand10K)



Correct (rand10K)

你需要做什么

你的任务是编写尽可能快且正确的 CUDA 渲染器实现。你可以采用任何你认为合适的方法，但你的渲染器必须遵守上面指定的原子性和顺序要求。不符合这两个要求的解决方案在作业第 3 部分将最多得到 12 分。我们已经给了你一个这样的解决方案！

一个好的起点是通读 `cudaRenderer.cu` 并确信它不满足正确性要求。特别是，看看 `CudaRenderer::render` 如何启动 CUDA 内核 `kernelRenderCircles`。（`kernelRenderCircles` 是所有工作的发生处。）为了直观地看到违反上述两个要求的效果，请使用 `make` 编译程序。然后运行 `./render -r cuda rand10k`，这应该显示带有 10K 个圆圈的图像，如上图底行所示。将此（不正确的）图像与运行 `./render -r cpuref rand10k` 生成的顺序代码生成的图像进行比较。

我们建议你：

1. 首先重写 CUDA 起始代码实现，以便在并行运行时逻辑上正确（我们建议采用不需要锁或同步的方法）
2. 然后确定你的解决方案存在的性能问题。
3. 此时作业的真正思考开始了.....（提示：在 `circleBoxTest.cu_inl` 中提供的圆圈-相交-盒子测试是你的朋友。我们鼓励你使用这些子程序。）

以下是 `./render` 的命令行选项：

Usage: ./render [options] scenename

有效的场景名称有: rgb, rgby, rand10k, rand100k, biglittle, littlebig, pattern, bouncingballs, fireworks, hypnosis, snow, snowsingle

程序选项:

-r --renderer <cpuref/cuda>	选择渲染器: ref 或 cuda (默认=cuda)
-s --size <INT>	使渲染的图像为 <INT>x<INT> 像素 (默认=1024)
-b --bench <START:END>	运行帧范围 [START,END) (默认 [0,1))
-f --file <FILENAME>	输出文件名 (FILENAME_XXXX.ppm)
-c --check	检查 CUDA 输出与 CPU 参考的正确性
-i --interactive	将输出渲染到交互显示
-? --help	显示此信息

检查代码: 为了检测程序的正确性, `render` 提供了一个方便的 `--check` 选项。此选项会同时运行参考 CPU 渲染器的顺序版本和你的 CUDA 渲染器, 然后比较生成的图像以确保正确性。也会打印出你的 CUDA 渲染器实现所花费的时间。

我们提供了总共五个圆圈数据集供你评估。然而, 为了获得满分, 你的代码必须通过我们所有的正确性测试。要检查你的代码的正确性和性能得分, 请在 `/render` 目录中运行 `./checker.py` (注意扩展名为 .py)。如果你在起始代码上运行它, 程序将打印如下表格, 并附上我们整个测试集的结果:

得分表:

场景名称	参考时间 (T_ref)	你的时间 (T)	得分
rgb	0.2321	(F)	0
rand10k	5.7317	(F)	0
rand100k	25.8878	(F)	0
pattern	0.7165	(F)	0
snowsingle	38.5302	(F)	0
biglittle	14.9562	(F)	0
		总得分:	0/72

注意: 在某些运行中, 你可能会为某些场景获得积分, 因为提供的渲染器的运行有时是非确定性的, 有时可能是正确的。但这并不改变当前 CUDA 渲染器总体上不正确的事实。

“参考时间”是你的当前机器上我们的参考解决方案的性能 (在提供的 `render_ref` 可执行文件中)。“你的时间”是你当前 CUDA 渲染器解决方案的性能, 其中 (F) 表示不正确的解决方案。你的成绩将取决于你的实现相对于这些参考实现的性能 (见评分指南)。

除了你的代码, 我们希望你提交一份清晰的高级描述, 说明你的实现是如何工作的, 以及你是如何得出这个解决方案的。具体地, 解释你尝试过的不同方法, 以及你是如何进行代码优化的 (例如, 你进行了哪些测量来指导你的优化工作?)。

你的工作中应该提到的方面包括:

1. 在你的报告顶部包括你和合作者的名字和 SUNet id。
2. 复制为你的解决方案生成的得分表, 并指定你运行代码的机器。
3. 描述你是如何分解问题的, 以及你是如何将工作分配给 CUDA 线程块和线程 (甚至可能是 warps) 。

4. 描述你的解决方案中发生同步的位置。
5. 描述你采取了哪些步骤来减少通信需求（例如，同步或主内存带宽需求）。
6. 简要描述你是如何得出最终解决方案的。你在此过程中尝试了哪些其他方法。它们有什么问题？

评分指南

- 作业的书面部分占 7 分。
- 你的实现占 72 分。每个场景的得分如下，每个场景的分数均等分为 12 分：
 - 每个场景 2 个正确性分数。
 - 每个场景 10 个性能分数（仅当解决方案正确时才能获得）。你的性能将相对于提供的基准参考渲染器的性能 T_{ref} 进行评分：
 - 如果时间 (T) 是 T_{ref} 的 10 倍，则不会获得性能分数。
 - 如果解决方案在优化解决方案的 20% 以内 ($T < 1.20 * T_{ref}$)，则获得满分性能分数。
 - 对于 T 值 ($1.20 T_{ref} \leq T < 10 * T_{ref}$)，你的性能得分将在 1 到 10 的范围内计算为：
 $10 * T_{ref} / T$ 。
- 你的实现的性能在班级排行榜上的表现占最后 6 分。排行榜的提交和评分细节将在后续的 Ed 帖子中详细介绍。
- 如果解决方案的性能显著高于要求，可以获得最多五分的额外加分（由教师决定）。你的书面报告必须清楚地解释你的方法。
- 如果实现了高质量的并行 CPU 渲染器，并且充分利用了所有核心和核心的 SIMD 向量单元，可以获得最多五分的额外加分（由教师决定）。可以使用任何工具（例如 SIMD 内在函数、ISPC、pthreads）。为了获得加分，你应该分析 GPU 和 CPU 解决方案的性能，并讨论实现选择差异的原因。

所以本项目的总分如下：

- 第 1 部分 (5 分)
- 第 2 部分 (10 分)
- 第 3 部分书面报告 (7 分)
- 第 3 部分实现 (72 分)
- 第 3 部分排行榜 (6 分)
- 可能的 **额外** 加分 (最多 10 分)

作业提示和建议

以下是从往年总结的一些提示和建议。请注意，有多种方法可以实现你的渲染器，并非所有提示都适用于你的方法。

- 本作业有两个可能的并行轴。一个轴是跨像素的并行，另一个是跨圆圈的并行（前提是对于重叠的圆圈要遵守顺序要求）。解决方案需要在计算的不同部分中利用这两种类型的并行性。
- 在 `circleBoxTest.cu_inl` 中提供的圆圈相交测试是你的朋友。鼓励你使用这些子程序。
- `exclusiveScan.cu_inl` 中提供的共享内存前缀和操作在本作业中可能对你有帮助（并非所有解决方案都选择使用它）。参见 [这里](#) 对前缀和的简单描述。我们提供了一个对 **2 的幂大小** 数组的独占前缀和的实现 **提供的代码不适用于非 2 的幂输入，并且还需要线程块中的线程数量为数组的大小。请阅读代码中的注释。**

- 如果你愿意，可以在实现中使用 [Thrust 库](#)。Thrust 对于实现优化的 CUDA 参考实现所需的性能不是必须的。有一种流行的解决问题的方法使用我们提供的共享内存前缀和实现。另一种流行的方法使用 Thrust 库中的前缀和例程。两者都是有效的解决方案策略。
- 渲染器中是否存在数据重用？可以做些什么来利用这种重用？
- 由于没有 CUDA 语言原语可以原子地执行图像更新操作，你将如何确保图像更新的原子性？一种解决方案是使用全局内存原子操作构建锁，但请记住，即使图像更新是原子的，更新也必须按所需顺序执行。**我们建议你首先考虑在并行解决方案中确保顺序，然后再考虑原子性问题（如果它仍然存在）。**
- 如果你有空闲时间，尽情制作你自己的场景吧！

捕捉 CUDA 错误

默认情况下，如果你访问数组越界、分配太多内存或引起其他错误，CUDA 通常不会通知你；相反，它会静默失败并返回错误代码。你可以使用以下宏（可以根据需要修改它）来包装 CUDA 调用：

```
#define DEBUG

#ifdef DEBUG
#define cudaCheckError(ans) { cudaAssert((ans), __FILE__, __LINE__); }
inline void cudaAssert(cudaError_t code, const char *file, int line, bool abort=true)
{
    if (code != cudaSuccess)
    {
        fprintf(stderr, "CUDA Error: %s at %s:%d\n",
            cudaGetErrorString(code), file, line);
        if (abort) exit(code);
    }
}
#else
#define cudaCheckError(ans) ans
#endif
```

注意，一旦你的代码正确，可以取消定义 DEBUG 以禁用错误检查，从而提高性能。

然后你可以包装 CUDA API 调用来处理它们返回的错误，如下所示：

```
cudaCheckError( cudaMalloc(&a, size*sizeof(int)) );
```

注意，你不能直接包装内核启动。相反，它们的错误将在你包装的下一个 CUDA 调用中捕获：

```
kernel<<<1,1>>>(a); // 假设内核引起错误！
cudaCheckError( cudaDeviceSynchronize() ); // 错误会在这行打印
```

所有 CUDA API 函数，如 `cudaDeviceSynchronize`、`cudaMemcpy`、`cudaMemset` 等都可以被包装。

重要提示： 如果之前某个 CUDA 函数发生错误，但没有被捕获，那么即使包裹了不同的函数，该错误也会在下一个错误检查中显示。例如：

```
...
line 742: cudaMalloc(&a, -1); // 执行, 然后继续
line 743: cudaCheckError(cudaMemcpy(a,b)); // 打印 "CUDA Error: out of memory at
cudaRenderer.cu:743"
...
```

因此, 在调试时, 建议你至少在自己编写的代码中包装**所有** CUDA API 调用。

(致谢: 改编自 [这个 Stack Overflow 帖子](#))

3.4 提交说明

请使用 Gradescope 提交你的作业。如果你与伙伴一起工作, 请记得在 Gradescope 上标记你的伙伴。

1. **请提交你的书面报告文件** `writeup.pdf`。
2. **请运行** `sh create_submission.sh` **生成一个 zip 文件并提交到 Gradescope**。注意, 这将会在你的代码目录中运行 `make clean`, 因此你需要再次运行 `make` 来运行你的代码。如果脚本错误提示“权限被拒绝”, 你应该运行 `chmod +x create\_submission.sh` 然后重试运行脚本。

我们的评分脚本将重新运行检查代码, 以验证你的得分与提交的 `writeup.pdf` 中的得分一致。我们可能还会尝试在其他数据集上运行你的代码, 以进一步检查其正确性。